Gran Quispe — Plan de desarrollo

Oct 20, 2026 · @Fernando Cardenas

Resumen ejecutivo

Gran Quispe es el juego de dados y faroleo cusqueño (el Quispe tradicional) recreado como aplicación web. Se construye para practicar y ampliar conocimientos de desarrollo, sumar una pieza sólida al portafolio, jugar con amigos, y más adelante explorar donaciones como forma de monetización.

Se diferencia de la app "Quispe" ya existente en el mercado por tres frentes: una identidad visual propia inspirada en Cusco (retablo ayacuchano, cantina nocturna), tres escenarios jugables, y un panel de configuración con variantes de reglas y bots — todo construido desde el día uno para escalar hacia un modo donde cada jugador participa desde su propio celular.

Concepto y reglas del juego

El Quispe se juega con un cubilete y dos dados. Cada jugador agita y mira su jugada sin mostrarla. La jugada mayor manda: un 4 y un 2 se lee 42, nunca 24. La escala va de 31 (la más baja) a 65. Los pares valen más y se leen 110, 120... hasta 160. La única jugada que supera todo, incluido el 160, es el 2 y 1, llamada "Quispe".

En cada turno, el siguiente jugador decide si cree o desconfía de lo que cantó el anterior. Si cree, pasa el cubilete sin ver los dados y debe cantar un número mayor. Mentir está permitido. Si desconfía, se destapa el cubilete: gana quien tenía razón.

Modos de juego

- Clásico QUISPE: cada ronda perdida suma una letra de la palabra QUISPE; quien la completa queda eliminado.
- Por vidas: cada jugador empieza con un número configurable de vidas.
- Castigo personalizado: igual al modo clásico, pero el castigo de cada ronda perdida es un texto libre que define quien configura la partida (ej. "toma un shot", "cuenta un secreto").

Variantes del panel de reglas

- Quispe vale doble: ganar con Quispe cuesta dos letras o dos vidas al perdedor.
- Reloj de presión: tiempo límite para decidir si se cree o se desconfía.
- Comodín de espía: una vez por partida, cada jugador puede ver uno de sus dos dados antes de decidir.

- Duelo final: con dos jugadores restantes, la partida pasa a muerte súbita.
- Bots: personalidades que mienten y desconfían con distinto criterio; se pueden sumar a cualquier partida normal.

Alcance del MVP

Requerimientos funcionales

- Modo "un solo celular": los jugadores se turnan el mismo dispositivo, con una pantalla intermedia de "pásale el celular" antes de revelar los dados de cada jugador.
- Panel de configuración de partida: modo de juego, variantes activas, número de jugadores (2 a 8), escenario y texto de castigo cuando aplica.
- Selector de idioma: español, inglés y términos en quechua.
- Bots que se pueden agregar a cualquier partida normal, con personalidad configurable.
- Guardado local de la partida en curso, con expiración automática pasado un tiempo corto, para no perder el progreso si se recarga el navegador sin que quede guardado indefinidamente.

Requerimientos no funcionales

- Responsivo, pensado primero para celular y luego para pantallas más grandes.
- Sin cuentas ni autenticación.
- Sin backend, base de datos ni servidor en el MVP; todo corre en el navegador.
- Accesibilidad básica: contraste legible entre texto y fondo, áreas táctiles de al menos 44 px.
- Arquitectura preparada para el modo online futuro: la lógica de reglas vive separada de la capa visual desde el día uno.

Identidad visual

Referencia validada: el wireframe de alta fidelidad hecho por el usuario en Claude Design (menú principal, mesa de juego, configuración) es la referencia visual oficial para el desarrollo; Claude Code debe construir sobre esa dirección de arte, no reinventarla.

Paleta de colores

Rol	Color	Hex
Fondo principal	Negro cálido	#120d0a
Madera / mesa	Marrón oscuro	#6b3f24

Rol	Color	Hex
Luz de foco	Ámbar dorado	#ffc46e
Acento principal	Naranja tostado	#f2a33a
Texto / UI	Crema	#f4ead7
Bordes / contornos	Negro	#000000
Marco retablo — franja 1	Rojo quemado	#c23a2b
Marco retablo — franja 2	Azul colonial	#2f5d9e
Marco retablo — detalle dorado	Dorado opaco	#f2c230
Letras QUISPE pendientes	Gris oscuro	#4a3a2f

Estilo gráfico: ilustración plana, contornos negros gruesos (3-4 px), sin degradados. Personajes tipo muñeco de madera pintado, con ropa regional cusqueña, exclusivos del escenario de cantina; el bar universitario y el paisaje andino son solo ambientación, sin personajes.

Tipografía: "Morelia" (la fuente de trazo artesanal que mencionaste) es una fuente de pago vendida en Creative Market, no está en Google Fonts y no viene lista para usarse en la web sin comprar su licencia y subirla como archivo propio. Para no bloquear el desarrollo, se recomienda arrancar con dos fuentes gratuitas de Google Fonts con el mismo espíritu artesanal y grueso que ya tiene el logo "Gran Quispe" del wireframe:

- **Títulos (display):** Luckiest Guy — gruesa, redondeada, con esa pinta de letrero pintado a mano de feria o cantina, muy cercana al look del logo ya validado.
- **Texto de interfaz (botones, listas, párrafos):** Fredoka — redondeada y amigable, legible en tamaños chicos, buena pareja para no competir con la fuente de título.

Si más adelante se compra la licencia de Morelia, se reemplaza solo la fuente de títulos sin tocar el resto del sistema, ya que queda referenciada como una sola variable de tipografía.

Stack tecnológico recomendado

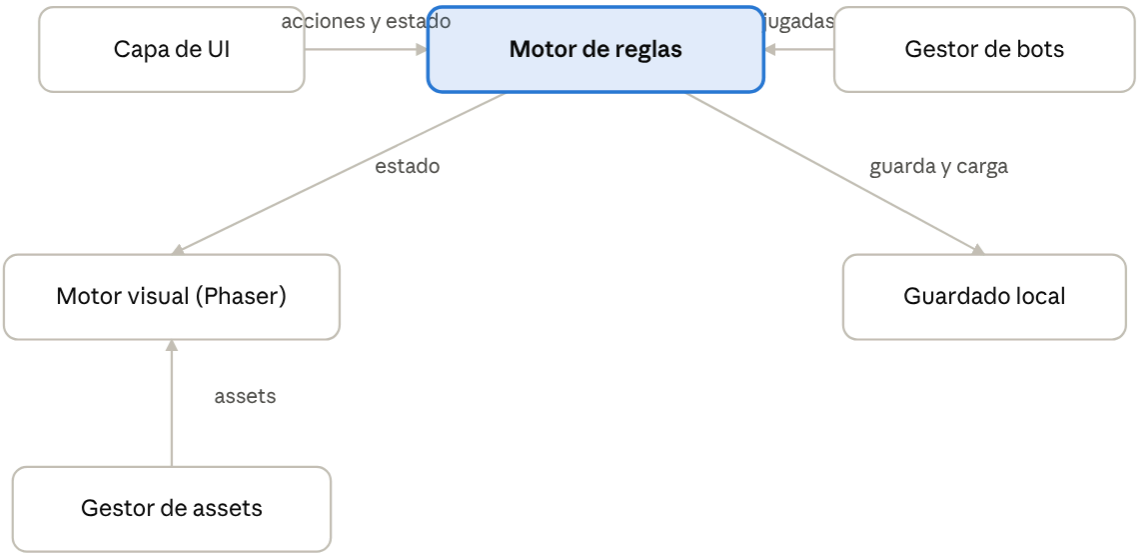
Capa	Elección	Por qué
Motor del juego	Vite + Phaser 3	Framework hecho para juegos 2D: maneja escenas (una por escenario), animaciones con rebote para el cubilete y los dados, audio y responsividad en celular de fábrica.

Capa	Elección	Por qué
Capa de UI	HTML y CSS simples sobre el canvas	Menú principal, panel de configuración, selector de idioma y pantalla de "pásale el celular" no necesitan el motor de juego; mantenerlos fuera de Phaser simplifica el mantenimiento.
Backend	Ninguno en el MVP	No hay cuentas ni partidas remotas que coordinar: todo corre en el navegador.
Base de datos	Ninguna en el MVP	No hay nada que persistir más allá del navegador de cada jugador.
Autenticación	Ninguna	Confirmado: no hace falta cuenta para jugar.
Guardado de partida	localStorage del navegador, con marca de tiempo	Permite "seguir donde quedó" sin backend; expira solo pasado un tiempo corto.
Hosting	Diferido	El usuario pidió primero tenerlo funcional en local; cuando se publique, un hosting estático gratuito (Vercel, Netlify, Cloudflare Pages) encaja con el resto de proyectos de bajo costo.

A futuro (modo online): Node.js con WebSockets (por ejemplo Socket.IO), reutilizando el mismo motor de reglas que corre en el MVP sin reescribirlo, porque desde el inicio queda separado de la interfaz.

Arquitectura y módulos del sistema

El motor de reglas es la única fuente de verdad del juego
Arquitectura de módulos: quién le habla a quién



arquitectura de módulos · 6 componentes

La UI y los bots solo leen el estado o envían acciones: el motor de reglas es la única fuente de verdad, y de él cuelgan el guardado local y, a través del motor visual, los assets.

Entidades de datos

Entidad	Campos clave
Jugador	nombre, avatar o inicial, letras QUISPE acumuladas o vidas restantes, esBot, personalidad (si es bot)
Partida	modo, variantes activas, escenario, lista de jugadores, turno actual, orden de giro
Ronda	jugada cantada, historial de lo dicho, quién tiró, resultado (creyó / desconfió, ganador)
Configuración de reglas	modo elegido, variantes on/off, número de jugadores, texto de castigo, idioma

Estructura de carpetas sugerida

Para que Claude Code la cree tal cual al iniciar el repositorio:

```
gran-quispe/
  src/
    engine/           # motor de reglas: máquina de estados, sin dependencias
visuales
      rules.js
      modes.js         # Clásico, Por vidas, Castigo personalizado
      variants.js       # Quispe doble, reloj de presión, comodín, duelo
final
      bots.js
    scenes/            # motor visual (Phaser), una escena por escenario
      CantinaScene.js
      BarScene.js
      AndinoScene.js
      shared/          # cubilete, dados, animaciones comunes a las tres
escenas
    ui/                # capa HTML/CSS fuera del canvas
      MainMenu.js
      RulesPanel.js
      PassDeviceScreen.js
    assets/
      images/<escenario>/
      audio/
      fonts/
    storage/
      localSave.js      # guardado local con expiración
    i18n/
      es.json
      en.json
      qu.json
  .claude/
    agents/
    skills/
  index.html
  vite.config.js
```

Configuración de Claude Code — agentes

Se crean como archivos Markdown en `.claude/agents/`, uno por archivo. Cada uno acota su propósito y, cuando aplica, las herramientas a las que tiene acceso, para que Claude Code delegue en el correcto según la tarea.

Subagente	Alcance	Cuándo delega Claude Code
`game-logic`	Solo `src/engine/`	Al tocar reglas, modos o variantes: valida que el motor de reglas siga sin depender de Phaser ni del DOM.
`phaser-scenes`	Solo `src/scenes/`	Al crear o ajustar animaciones, escenas o la lógica visual del cubilete y los dados.
`code-reviewer`	Lectura de todo el repo, sin permiso de escritura	Antes de dar por cerrada cualquier tarea: revisa rendimiento, código duplicado y que la separación motor/vista se mantenga.

Cada archivo lleva `name`, `description` (la frase que activa la delegación automática) y el prompt del rol. Por ejemplo, `game-logic.md` empieza con: "Eres responsable únicamente del motor de reglas de Gran Quispe. Nunca importas Phaser ni tocas el DOM. Toda función es pura y testeable."

Configuración de Claude Code — skills

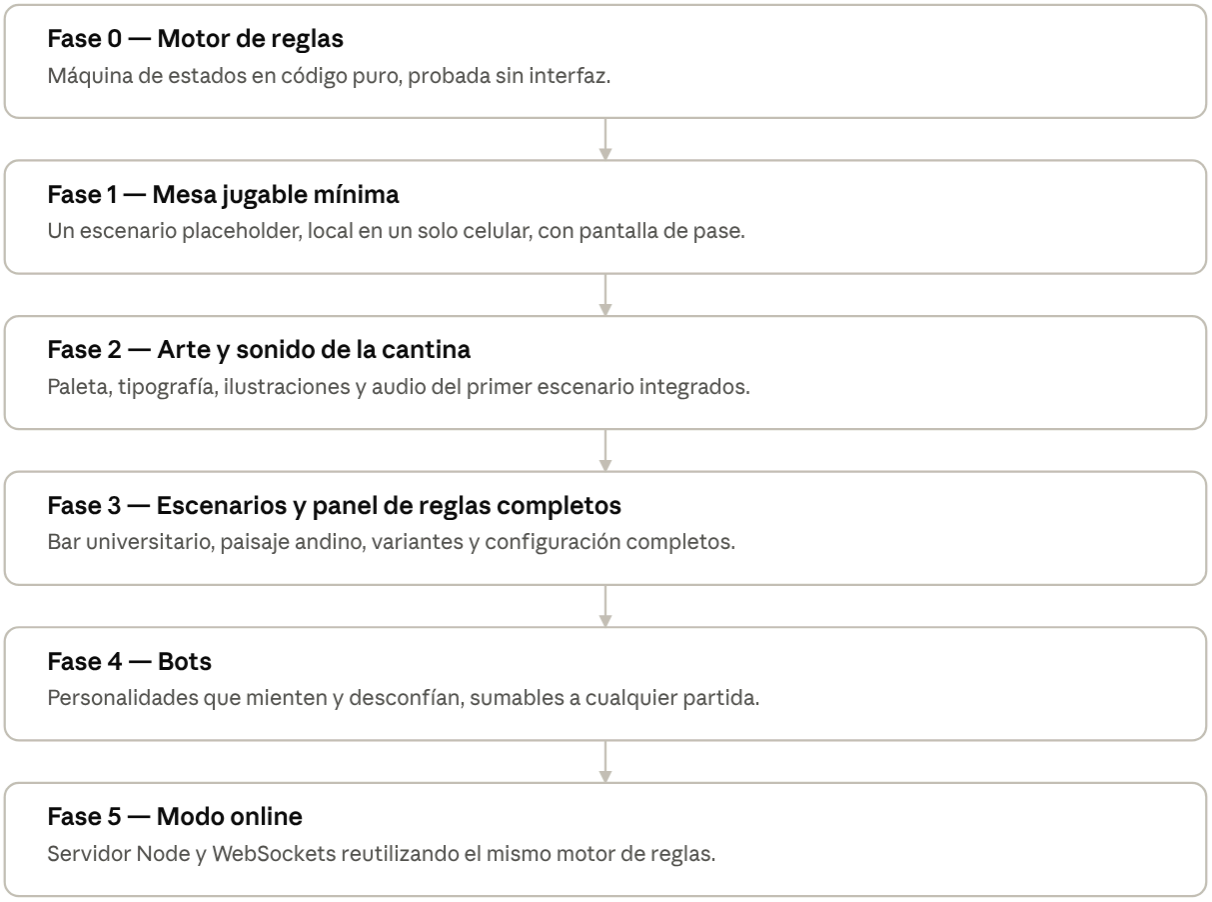
Se crean en `.claude/skills/<nombre>/SKILL.md`, disponibles para cualquier sesión de Claude Code en el proyecto.

Skill	Contenido	Se activa cuando
`identidad-visual`	La paleta de colores exacta de esta sección, la pareja tipográfica (Luckiest Guy + Fredoka), tamaños de sprite esperados y la convención de nombres de archivos de `src/assets/`	Se crea o integra cualquier gráfico, color o fuente nuevos
`codigo-limpio-quispe`	La regla de separación motor/vista de la sección de arquitectura, convenciones de nombres, y cómo se agrega una nueva variante al panel de reglas sin tocar el motor visual	Se escribe o revisa cualquier código del proyecto

Ambos skills los arma Claude Code directamente dentro del repositorio al iniciar el proyecto, usando el contenido de este documento como fuente.

Orden de desarrollo sugerido

Seis fases, de la lógica pura al modo online
Orden de desarrollo sugerido



plan de fases · 6 etapas

Cada fase deja algo jugable o probable antes de sumar la siguiente; el modo online llega al final porque reutiliza el motor de reglas ya probado, sin rehacerlo.

Riesgos y alertas detectados

Riesgo	Por qué importa	Cómo se mitiga
Nombre ya usado	Ya existe una app "Quispe: Juego de Dados" en Play Store	Se adopta "Gran Quispe" como nombre propio, ya definido
Información oculta en un solo celular	Si no hay pantalla intermedia, el siguiente jugador ve los dados del anterior al pasar el teléfono	Pantalla de "pásale el celular" obligatoria entre turnos, ya contemplada en el wireframe

Riesgo	Por qué importa	Cómo se mitiga
Contenido de alcohol en el modo castigo	Tiendas de apps y redes de anuncios pueden restringir o subir de edad contenido que incentive tomar	El castigo queda como texto libre editable por quien configura, no como identidad fija del juego
Mezclar lógica y vista	Si las reglas quedan acopladas a Phaser, el futuro modo online obliga a reescribir el motor	El motor de reglas vive aislado en `src/engine/`, sin importar Phaser ni el DOM, reforzado por el skill `codigo-limpio-quispe`